

Type Inference & Unification

Algorithm

1. Assign fresh type vars to every parameter.
2. Recurse down AST as in normal type checking.
3. On the way back up, **unify** types.

Unification

Base cases: type var with type $\rightarrow$ mapping; same type $\rightarrow \emptyset$; constructor mismatch $\rightarrow$ FAIL.

Recursive: $\text{unify}(T_1 * T_2, S_1 * S_2):$ $\text{sub1} = \text{unify}(T_1, S_1)$; apply sub1 to T_2, S_2 ; $\text{sub2} = \text{unify}(T_2', S_2')$; combine. **Must apply sub from left before right.**

Concrete examples:

- $\text{unify}(\text{int} * 'a, \text{int} * \text{string}) \rightarrow \{a \rightarrow \text{string}\}$
- $\text{unify}(\text{int} \rightarrow 'a, 'b * \text{string}) \rightarrow \text{FAIL}$ (arrow vs tuple)
- $\text{unify}('a * 'a, \text{int} * \text{string})$: left gives $a \rightarrow \text{int}$; apply to right: $\text{unify}(\text{int}, \text{string}) \rightarrow \text{FAIL}$

Occurs Check

When mapping $'b$ to T , $'b$ cannot appear inside T .

$\text{fun } f(x) = f: f: a \rightarrow b$, body is f itself ($a \rightarrow b$). Unify $'b$ with $a \rightarrow b \rightarrow$ infinite type $\rightarrow$ **FAIL**.

Workaround: $\text{datatype } 'a \text{ hungry} = F \text{ of } 'a \rightarrow 'a \text{ hungry}$. Then $\text{fun } f(x) = F(f) : 'a \rightarrow 'a \text{ hungry}$. SML datatypes allow recursive types.

Worked: Type Inference for len

```
fun len list = case list of [] => 0 | y::z => 1 + len z
Assign: list:'a, return:'b, y:'c, z:'d. len : 'a -> 'b.
From len(z): unify 'a with 'd -> 'a='d.
From 1+len z: plus needs int*int -> int, so 'b=int.
From pattern y::z: cons has type 'e*'e list -> 'e list, so 'e='c, 'd='c list.
From nil: type 'f list, unify with 'c list -> 'f='c.
Result: len : 'c list -> int.
```

Practice Final Q1

```
fun f(w,x) = case x of [] => [] | y::z => w(y)::f(w,z)
Assign w:'a, x:'b. Unification table:
x with [] -> 'b = 'f list. (y,z) with cons arg -> 'd='h, 'e='h list. x with
cons result -> 'f='h. w as fn taking y -> 'a = 'h -> 'i. (w(y),f(w,z))
with cons -> 'i='j, 'c='j list. 1st case = 2nd case -> 'k='j.
Result: f : ('h -> 'j) * 'h list -> 'j list (this is map).
fun f(x) = f fails the occurs check.
```

Options Gotcha

$\text{case } \dots \text{ of } \dots \Rightarrow \text{NONE} \mid \dots \Rightarrow 42$ FAILS: unify $'a$ option with $\text{int} \rightarrow$ tycon mismatch. Fix: use **SOME 42** so both branches are int option.

Subtyping

Covariance: read $\Rightarrow$ same direction. **Contravariance:** write $\Rightarrow$ opposite. **Invariant:** read+write $\Rightarrow$ no subtyp-

ing.

Functions: covariant in return, contravariant in arg.

If $\text{Lion} <: \text{Cat}$: $\text{Lion list} <: \text{Cat list}$ (covariant); $\text{Lion ref} \not<: \text{Cat ref}$ (invariant – can write a Frog into Cat ref, then read expecting Lion); $\text{Cat} \rightarrow \text{int} <: \text{Lion} \rightarrow \text{int}$ (contra in arg).

SML: $'a = \text{any type}$, $''a = \text{equality types}$. Complex constraints $\rightarrow$ **functors**.

MIPS Registers & Calling Convention

Reg	Name	Purpose
r0	zero	Hardwired to 0
r1	AT	Assembler temp (pseudo-instrs)
r2-3	v0-v1	Return values
r4-7	a0-a3	Arguments (4 on stack)
r8-15	t0-t7	Caller-saved temps
r16-23	s0-s7	Callee-saved
r24-25	t8-t9	Caller-saved temps
r29	sp	Stack pointer
r30	fp	Frame pointer
r31	ra	Return address

Function Prolog

- (1) $\text{SP} -= \text{frame_size}$
- (2) Save old FP
- (3) $\text{FP} = \text{SP} + \text{frame_size}$
- (4) Save S regs used
- (5) Save RA (only if non-leaf – leaf = calls nobody)

Function Epilog

- (1) Restore S regs
- (2) Restore RA
- (3) Reset SP via FP
- (4) Restore old FP
- (5) $\text{jr } \$ra$

Caller's Responsibilities

Before: save T regs needed, args to a0-a3 (extras on stack), jal. **After:** restore T regs.

Why Frame Pointer Matters

FP is **fixed**; SP changes with `alloca()`, VLAs, nested call args. Vars: FP – offset (stable) vs SP + offset (varies). Debuggers use FP chain. **Can omit FP** when no dynamic stack alloc.

Static Links & Escaping Variables

Each frame stores pointer to **statically enclosing function's** frame. Follow N static links to access variable N levels out. In Tiger: SL passed as implicit first arg (a0). $\text{SL} \neq \text{old FP}$: FP follows dynamic call chain; SL follows lexical nesting.

Static Link Rules + Practice Final Q2

Calling child: pass own FP. **Calling sibling/recursive:** pass own SL (MEM(FP)). **Calling outer function:** follow SL chain outward. Nesting: top has odd, f; inside f: g, h. $f \rightarrow g$: **FP** (child). $g \rightarrow h$: **MEM(FP)** (sibling). $g \rightarrow g$: **MEM(FP)** (recursive). $h \rightarrow f$: **MEM(MEM(FP))** (2 up). $g \rightarrow \text{odd}$: **MEM(MEM(FP))** (2 up). **Deep nesting:** foo inside h inside g inside f. To access a from f's frame in foo: follow 3 static links ($\text{foo} \rightarrow h \rightarrow g \rightarrow f$). Always exactly 3 regardless of recursion depth.

Escaping Variables

Tiger: used in nested function $\rightarrow$ must live in stack frame. C: address taken. **Escape analysis:** recurse AST, track declaration depth, mark escaping if referenced deeper.

Intermediate Representation (IR)

$L \text{ langs} \times I \text{ ISAs} \rightarrow L \cdot I \text{ compilers}$. With IR: $L + I$.

Statements: $\text{SEQ}(s_1, s_2)$, $\text{LABEL}(l)$, $\text{JUMP}(e, \text{labels})$, $\text{CJUMP}(\text{relop}, e_1, e_2, \text{lt}, \text{lf})$ [**two targets**], $\text{MOVE}(\text{dst}, \text{src})$ [$\text{dst} = \text{TEMP}$ or MEM], $\text{EXP}(e)$.

Expressions: $\text{BINOP}(\text{op}, e_1, e_2)$, $\text{MEM}(e)$ [in MOVE $\text{dst} = \text{store}$; else = load], $\text{TEMP}(t)$, $\text{ESEQ}(s, e)$, $\text{NAME}(l)$, $\text{CONST}(n)$, $\text{CALL}(f, \text{args})$.

Ex / Nx / Cx Wrappers & Conversions

Ex=expression, **Nx**=statement, **Cx**=condition function ($|\text{bl}| * |\text{bl}| \rightarrow \text{stm}$).

unEx(Cx(f)): alloc r; $\text{MOVE}(r, 1)$; $f(\text{tl}, \text{fl})$; $\text{LABEL } \text{fl}$; $\text{MOVE}(r, 0)$; $\text{LABEL } \text{tl}$; $\text{result} = \text{ESEQ}(\dots, \text{TEMP } r)$.

unCx(Ex(e)): $\text{CJUMP}(e \neq 0, \text{true}, \text{false})$.

unEx(Nx(s)): $\text{ESEQ}(s, \text{CONST } 0)$.

Practice Final Q3 – IR Translation

```
x -> InReg t1, y -> InReg t2, z -> InFrame -4, a -> InReg t3.
y := 1 + x: MOVE(TEMP t2, BINOP(PLUS, CONST 1, TEMP t1))
a[y] := f(z): MOVE(MEM(BINOP(PLUS, TEMP t3,
BINOP(TIMES, TEMP t2, CONST 4))), CALL(LABEL f,
[MEM(BINOP(PLUS, FP, CONST -4))]))
Record r.z: MEM(BINOP(PLUS, TEMP r, CONST 8)) - z is 3rd
field, offset 8.
```

Translating Control Flow

If-Then-Else

```
c' = unCx(cond), t' = unEx(then), e' = unEx(else)
r = newTemp, lt/lf/le = newLabels
ESEQ(SEQ(c' (lt, lf), LABEL lt,
MOVE(TEMP r, t'), JUMP(NAME le, [le])),
LABEL lf, MOVE(TEMP r, e'),
LABEL le), TEMP r)
```

While Loop (Optimized – 1 branch/iter)

```
JUMP(NAME L_test, [L_test])
L_body: ... body ...
L_test: CJUMP(cond, L_body, L_done)
L_done:
```

Naive has 2 branches/iter ($\sim 2M$ for $1M$ iters). Optimized: $1M + 1$.

Break: JUMP(NAME(done.label)). Track with label option.

For Loop – Max-Int Edge Case

Tiger inclusive bounds. If high=maxint, naive i++ overflows → infinite loop.

```
i:=low; h:=high
if i<=h goto L_body else L_done
L_body: ... body ...
  if i<h then (* STRICT, BEFORE incr *)
    i:=i+1; goto L_body
  else goto L_done
L_done:
```

Arrays

Layout: [size — elem0 — elem1 — ...]. Return ptr to elem0; size at MEM(arr-4).

Create: malloc (size+1)*4, store size at offset 0, incr ptr by 4, fill values.

Index: MEM(base + idx*4). Eval idx into temp first (avoid dup side effects).

Bounds: $0 \leq \text{idx} < \text{MEM}(\text{arr} - 4)$. If out of bounds, jump to exit label.

Records

malloc, MOVE each field to MEM at offset. Access: MEM(base + offset) – offset known at compile time. No bounds check.

Switch/Case

If-else $O(n)$; binary search $O(\log n)$; **jump table** $O(1)$ for dense cases (array of labels, indirect jump); hybrid for mixed. JUMP takes exp*label list for jump tables.

Critical Rule

Never reuse an IR sub-tree. Dup labels=asm error; dup side effects=bugs.

Purifying & Linearizing

Removing ESEQ

- $\text{ESEQ}(s1, \text{ESEQ}(s2, e)) \rightarrow \text{ESEQ}(\text{SEQ}(s1, s2), e)$
- $\text{BINOP}(+, \text{ESEQ}(s, e1), e2)$: if independent $\rightarrow \text{ESEQ}(s, \text{BINOP}(+, e1, e2))$. Else $\rightarrow \text{ESEQ}(\text{SEQ}(\text{MOVE}(t, e1), s), \text{BINOP}(+, t, e2))$. Extra temp increases register pressure but is always safe.

Hoisting Function Calls

$f(g(x), h(y))$ clobbers shared regs (a0, v0). Fix: $t1=g(x); t2=h(y); \text{result}=f(t1, t2)$. L-to-R in Tiger.

Alias Analysis

Yes/No/Maybe. **Undecidable.** (1) Conservative: all memory may interfere. (2) Type-based: different types

can't alias (GCC -fstrict-aliasing). (3) Allocation-site: different malloc sites → different objects. Never false negatives.

Linearizing

False branch of every CJUMP falls through. **DFS** $\gg$ BFS (fewer jumps). **notRel**: negate relops when true branch needs to fall through.

Instruction Selection

Maximal Munch

- (1) Match **largest** IR subtree to one asm instruction.
- (2) Recurse on remaining children.
- (3) Emit instruction, return result register.

Worked: a[i

```
= (z.field3 + 27) * 3] IR: MOVE(MEM(+(a, *(i, 4))),
*(+(MEM(+(z,8)), 27), 3))
li t100, 4 (CONST 4)
mul t101, i, t100 (i*4)
add t102, a, t101 (a + i*4)
lw t103, 8(z) (MEM(+(z,8)) – load with offset!)
addi t104, t103, 27 (add immediate!)
li t105, 3
mul t106, t104, t105
sw t106, 0(t102) (store word)
Key patterns: MEM(+(e,CONST c)) → single LW with offset.
+(e,CONST c) → ADDI.
```

Output Data Types

OPER: asm string + src/dst lists + jump targets. **MOVE**: reg-to-reg (kept separate for coalescing). **LABEL**: marker.

Jump list: NONE=fall through; SOME(list)=targets; SOME($\square$)=end of fn (JR RA).

JAL dsts: list ALL caller-saved (t0–t9, a0–a3, v0, ra) because callee may clobber them.

Liveness Analysis

Live: value may be read before overwritten. False positives (say live when dead) are safe.

def(B): vars written. **use(B)**: vars read before any def in B.

Liveness Equations

```
live-in(B) = (live-out(B) – def(B))  $\cup$  use(B)
live-out(B) =  $\bigcup_{S \in \text{succ}(B)} \text{live-in}(S)$ 
Backward, union, least fixed point (start  $\emptyset$ ).
```

Worked: Instruction-Level Liveness

```
Block: x=q+a; y=c*3; z=5. live-out={x, y, u}.
After z=5: {x, y, u} (kill z, not in set; gen nothing)
After y=c*3: kill y, gen c  $\rightarrow$  {x, c, u}
After x=q+a: kill x, gen q, a  $\rightarrow$  {q, a, c, u} = live-in
```

Worked: Liveness with Loops

Key insight from vampire/zombie example: liveness propagates backwards through loop back-edges. First pass may miss variables; second pass propagates them through the cycle. “Zombies propagate everywhere” – variables used inside a loop become live throughout it. “Vampires die at dawn” – killed (defined) variables stop being live at that point.

Convergence: finite vars + sets only grow $\rightarrow$ must converge. Process blocks in **reverse order** for efficiency.

Infeasible Paths

If $x < 7$ true but $x \geq 9$ impossible, liveness can't detect this $\rightarrow$ false positives. Improving precision approaches undecidability.

Register Allocation

Interference Graph

Nodes=temp. Solid edge: live at same time. Dashed: move-related. Build: for each write of t, add interference with all live-out temps (except t). For MOVE: move edge instead.

k = physical registers. Goal: k -color the graph. NP-complete but heuristic works well.

Algorithm: Simplify $\rightarrow$ Coalesce $\rightarrow$ Freeze $\rightarrow$ Spill

1. **Simplify**: degree $< k$ AND no move edges $\rightarrow$ remove, push stack. Move edges do NOT count toward degree.
2. **Coalesce**: merge move-related nodes.

Coalescing Heuristics (either $\Rightarrow$ safe)

Briggs: Combined node has $< k$ neighbors of non-trivial degree ($\geq k$). Trivial-degree neighbors can always be simplified later.

George (A into B): Every neighbor t of A either already interferes with B (no new constraint), or has trivial degree ($< k$, simplifiable). Check each direction separately – asymmetric!

3. **Freeze**: give up on a move edge (prefer where ≥ 1 node is trivial degree).

4. **Potential Spill**: pick node, remove, mark with star.

5. **Select**: pop stack, assign colors. Simplified=guaranteed. Potential spills may fail $\rightarrow$ actual spill $\rightarrow$ insert loads/stores, **restart from scratch**.

Worked: Coalescing Check for C and D

George C $\rightarrow$ D: C interferes with M (already interferes with D? **YES**), C interferes with B (already interferes with D? **YES**). All neighbors OK $\rightarrow$ **safe to coalesce**.

George D $\rightarrow$ C: D interferes with K (interferes with C? **NO**). Trivial degree? **NO** $\rightarrow$ **fails**.

One direction passed $\rightarrow$ proceed with coalesce.

Practice Final Q4 – 3-reg machine

(1) Simplify b (trivial degree, no move edges). (2) Coalesce p,q (move-related, safe). (3) Simplify pq. (4) Freeze x,c move edge (can't coalesce). (5) Simplify x,y,a,z,c.

Rebuild: $c=r1, z=r2, a=r3, y=r1, x=r2, pq=r2, b=r3$.

Pre-colored Temps

Fixed regs (temp100=a0). All interfere with each other. **Never simplified/spilled**. Base case: only pre-colored remain.

Temp live across JAL $\rightarrow$ interferes with all caller-saved $\rightarrow$ naturally gets s-register.

S-reg strategy: Try 0 s-regs; if fails, try 1, 2, etc.

Data Flow Analysis Framework

Master Table				
Analysis	Dir	Meet	Init	Drives
Avail Exprs	Fwd	$\cap$ pred	Greatest	CSE
Reaching Defs	Fwd	$\cup$ pred	Least	const/copy prop
Liveness	Bwd	$\cup$ succ	Least	reg alloc
Very Busy	Bwd	$\cap$ succ	Greatest	code hoisting

$\cap \Rightarrow$ greatest fixed pt (start full). $\cup \Rightarrow$ least fixed pt (start $\emptyset$).

Available Expressions

x op y is **available** if computed on **ALL** paths and operands unmodified.

$AE\text{-in}(B) = \bigcap_{P \in \text{pred}} AE\text{-out}(P)$, $AE\text{-out}(B) = (AE\text{-in}(B) - \text{Kill}) \cup \text{Gen}$

Gen: exprs computed (operands not killed in B). **Kill:** exprs using vars defined in B.

Worked: Why Greatest Fixed Point

Block 1: $a = x * y \rightarrow$ loop (doesn't modify x, y) $\rightarrow$ Block 2: $b = x * y$.
Wrong (start $\emptyset$): Loop merge: $\{x * y\} \cap \{\} = \{\}$. $x * y$ never available. WRONG.
Right (start full): Loop merge: $\{\text{everything}\} \cap \{x * y\} = \{x * y\}$. Propagates correctly. $x * y$ IS available at Block 2. Cycles with intersection need greatest fixed point. Starting $\emptyset$ kills everything through cycles.

Reaching Definitions

Def “ $x := \dots$ ” reaches use if path exists with no redef of x .

$RD\text{-in}(B) = \bigcup_P RD\text{-out}(P)$, $RD\text{-out}(B) = (RD\text{-in}(B) - \text{Kill}) \cup \text{Gen}$

Gen: defs in B. **Kill:** all other defs of same var.

Worked: Reaching Defs Applications

Constant prop: Only reaching def is $x=0$, have $a = x + q \rightarrow a = 0 + q = q$.
Copy prop: Only reaching def is $x=y \rightarrow$ replace uses of x with y . Then $x=y$ is dead code.
Uninit var: No defs reach a use $\rightarrow$ variable may be uninitialized.
Dead code: Def reaches no use and no side effects $\rightarrow$ delete.

Practice Final Q5 – Reaching Defs

a in instr 4: defs **1, 8**. a in instr 8: defs **1, 8, 13**. b in instr 6: defs **2, 11**.

Very Busy Expressions

Definitely computed on all forward paths before operand modified. $VBE\text{-out}(B) = \bigcap_{S \in \text{succ}} VBE\text{-in}(S)$
Drives: code hoisting (precompute earlier). Also LICM safety check. Caveat: moving code earlier can change exception behavior.

Live Range Splitting

Label all defs ($D0, D1, \dots$) and uses ($A, B, \dots$) of a variable. For each use, find reaching defs. Build graph connecting uses to their reaching defs. Each **connected component** = separate live range $\rightarrow$ assign different temps $\rightarrow$ lower interference degree.

Example: Two for-loops with i . Defs $D0, D1$ (1st loop), $D2, D3$ (2nd). Uses $A-D$ reached by $\{D0, D1\}$, uses $E-H$ by $\{D2, D3\}$. Two disjoint components $\rightarrow$ split i into i and i' .

Practice Final Q5 – Matching

CSE $\rightarrow$ Avail Exprs. Fwd/union $\rightarrow$ Reaching Defs. Def/use web $\rightarrow$ Reaching Defs. Bwd/intersect $\rightarrow$ Very Busy.

Compiler Optimizations

Worked: Quicksort Partition Optimization

Naive code computes $4*i$ five times, $4*j$ multiple times.
1. CSE: $4*i$ computed in B2 as t2. Check: is $4*i$ available at other uses? i not changed between B2 $\rightarrow$ B5 $\rightarrow$ YES, reuse t2. Eliminates 4 multiplies. Same for $4*j$ (reuse t4) and $4*\text{high}$ (reuse t1).
2. Move elimination: After CSE, $t6=t2$ (was $4*i$). Only reaching def $\rightarrow$ replace all uses of t6 with t2. Delete $t6=t2$ (dead code).
3. Reuse loads: $a[t2]$ loaded in B2 (into t3). No stores between B2 and B5 $\rightarrow$ reuse t3. B5 swap goes from 9 instructions to 3 (just stores + branch).
4. Alias gotcha: Can't reuse $a[t1]$ ($a[\text{high}]$) in B6 because swap in B5 writes to $a[t2]$ and $a[t4]$ which *might* equal $a[t1]$. Compiler can't prove they don't.
5. Induction var elim: i only changes by $i=i+1$ and only used as $4*i \rightarrow$ replace with $t2+=4$. Similarly j : $t4-=4$.
6. Overflow caveat: $4*i \geq 4*j \neq i \geq j$ in fixed-width! But $4*i$ is an array index, so overflow would segfault first $\rightarrow$ safe to assume no overflow.

LICM: All defs reaching operands from outside loop $\rightarrow$ hoist.

Worked: LICM Danger

for(i) for(j) total += array[i][j]. array[i] is loop invariant w.r.t. inner loop $\rightarrow$ hoist to $p = \text{array}[i]$.
Danger: If $m=0$ and array is null, original never executes array[i]. Hoisting causes segfault in working code!
Fix: while $\rightarrow$ if(cond){do-while}. do-while executes at least once $\rightarrow$ array[i] is a very busy expression at loop entry $\rightarrow$ hoisting is safe.

Strength reduction: $4*i \rightarrow i \ll 2$. Mul by 3 $\rightarrow$ shift+add.

Domination & Loop ID

A **dominates** B : all paths from start to B pass through A . Self-dominates.

$\text{Dom}(B) = \{B\} \cup \bigcap_{P \in \text{pred}} \text{Dom}(P)$. Entry: {entry}. Intersection $\Rightarrow$ greatest fixed pt (init universal).

idom(B): closest strict dominator. Dom tree: path root $\rightarrow B$ = all dominators.

Back edge: $E \rightarrow S$ where S dominates $E \rightarrow$ natural loop. NOT a loop if target doesn't dominate source.

Worked: 10-Node Domination

$\text{Dom}(0) = \{0\}$, $\text{Dom}(1) = \{0, 1\}$, $\text{Dom}(2) = \{0, 2\}$, $\text{Dom}(3) = \{0, 2, 3\}$,
 $\text{Dom}(4) = \{0, 4\}$, $\text{Dom}(5) = \{0, 4, 5\}$, $\text{Dom}(6) = \{0, 4, 5, 6\}$,
 $\text{Dom}(7) = \{0, 4, 5, 7\}$, $\text{Dom}(8) = \{0, 4, 5, 8\}$, $\text{Dom}(9) = \{0, 4, 5, 8, 9\}$.
 Back edges: $3 \rightarrow 3$ (self-loop), $8 \rightarrow 4$ (4 dom 8).
 $4 \rightarrow 1$ is **NOT** a loop: 1 doesn't dominate 4 (path $0 \rightarrow 2 \rightarrow 3 \rightarrow 4$ bypasses 1).
 idom tree: 0 root; idom(1)=0, idom(2)=0, idom(4)=0; idom(3)=2; idom(5)=4; idom(6)=5, idom(7)=5, idom(8)=5; idom(9)=8.

Garbage Collection

Garbage: unreachable memory. Undecidable precisely; approximate by reachability.

Reference Counting

Worked: Code for p

(1) if $q \neq \text{null}$: temp = q.refcount; temp++; q.refcount = temp
 (2) if $p \neq \text{null}$: temp = p.refcount; temp--; if temp == 0: recursive-Free(p); else p.refcount = temp
 (3) $p = q$
 One move becomes ~ 3 branches, 2 loads, 2 stores, 2 arith ops. In MT: atomic ops (~ 50 cycles each).

Pros: Simple, incremental. **Cons:** Slow ($\sim 10x$), **can't collect cycles** ($A \rightarrow B \rightarrow A$ stays refcount ≥ 1 forever).

Mark and Sweep

Root set: regs, stack, globals. **Mark:** DFS, mark reachable (mark bit=visited). **Sweep:** walk heap linearly using size headers; unmarked $\rightarrow$ free list; marked $\rightarrow$ clear bit.

Pointer ID: object header **bitmasks** (which fields are ptrs). Stack frame bitmasks. PC-keyed register bitmasks (table: return addr $\rightarrow$ 32-bit reg bitmask).

Cost: $O(\text{live} + \text{dead})$. Stop-the-world.

Stop and Copy

Worked: Stop-and-Copy

Heap split into active/inactive halves. Allocate via **bump pointer** (super fast).
 When full: DFS from roots. Copy D to inactive, leave **forwarding ptr**. D refs B $\rightarrow$ copy B, leave fwd ptr. B refs A $\rightarrow$ copy A. G refs B $\rightarrow$ B already copied (fwd ptr) $\rightarrow$ link to copy. C, E, F are dead – **never touched**.
 Swap halves. Dead objects cost nothing!

Pros: $O(\text{live})$ only, no fragmentation, fast alloc, good locality. **Cons:** wastes half heap.

Generational GC

Infant mortality: most objects die young.

Gen 0 (~256KB, fits L2): stop-and-copy, very frequent, ~95% garbage. **Gen 1** (~4MB): mark-and-sweep, less frequent. **Gen 2** (~6GB): rare. Promote after surviving N collections (2-bit counter).

Old→young ptrs: mprotect() marks old pages read-only; writes trigger SIGSEGV to record dirty pages for root set during young-gen collection.

Practice Final Q7 – GC

3 advantages: Fast alloc (S&C), locality (S&C), no double frees, no dangling ptrs, no leaks, reduced dev time.

Tradeoffs: RefCount=high per-op overhead, 1 word/obj, can't do cycles. M&S= $O(\text{live}+\text{dead})$, 1 word/obj, fragmentation. S&C= $O(\text{live})$, **wastes half heap**, compacts.

Pointer ID: (1) Compiler type info/bitmasks, (2) Conservative (can't move objects), (3) Tagging (steal 1 bit/word).

Compiling OO Languages

Polymorphism: (1) Ad-hoc/overloading → name mangling. (2) Parametric: C++ specialization (sep code/-type) vs Java erasure (compile as Object). (3) Subtype → vtables.

VTables & Dynamic Dispatch

Each obj has **one ptr** to shared class vtable. All instances share one vtable.

Worked: Dynamic Dispatch

```
v = load(p+0)    % vtable ptr
f = load(v+offset) % method ptr from vtable
a0 = p; jalr f    % pass this, indirect call
Cost: 2 extra loads vs static dispatch.
C++ requires explicit virtual. Java: everything is dynamic (prevents forgot-virtual bugs).
```

Sub-object Rule

Subclass must contain superclass at **same offsets**. Cat extending Animal: [vtable — name — age — laziness] — first 3 fields = valid Animal.

Multiple Inheritance

Sub-objects stacked: [cat.vtable — name — age — laziness — robot.vtable — robot_name — model]. **Two vtable ptrs.** Casting to non-primary parent: $f(p + \text{offset})$ (pointer adjustment).

Thunks: If CyborgCat overrides Robot::m2, secondary vtable entry points to fixup code that subtracts offset from **this** before calling actual method.

Diamond / Virtual Inheritance

Two parents sharing grandparent → two copies of grandparent fields. Virtual inheritance: shared base at END, accessed via **offset in vtable** → extra indirection cost. Only one copy of shared fields.

JIT (Java)

javac→bytecode→interpret first→compile hot code to native→profile-guided recompilation.

Miscellaneous Exam Topics

procEntryExit 1/2/3: prolog/epilog, JR RA (SOME($\square$)), **view shift:** move args from a0–a3/stack into temps/frame slots IR expects (e.g., MOVE(t100,a0); SW(a1,FP–4)).

Instr scheduling: Reorder independent instrs to avoid pipeline stalls. **Loop unrolling** enlarges scheduling scope. Needs alias analysis to interleave loads/stores from different iterations.

CPS: Confirmed NOT on exam.

Undecidability pattern: Alias analysis, precise liveness, optimal reg alloc: all undecidable/NP-hard. Approximate safely: liveness false pos (extra pressure), alias false pos (miss opts), reg alloc may fail → spill.

Optimization	Driven by
CSE	Available Expressions
Const/Copy propagation	Reaching Definitions
Dead code elimination	Reaching Defs + Liveness
Live range splitting	Reaching Definitions
Register allocation	Liveness
Code hoisting	Very Busy Expressions
LICM safety	Very Busy + Domination